1. Data Collection:

```
# Import libraries
import pandas as pd
import matplotlib pyplot as plt
import seaborn as sns
from itertools import combinations
from collections import Counter
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
# load the dataset in dataframe
df = pd.read_csv(r"C:\Users\Public\Downloads\SuperMart grorocery sales.csv")
```

2. Data Cleaning & Pre-Processing:

```
# Convert month column to datetime format
df['Order_Date'] = pd.to_datetime(df['Order_Date'], format='%Y-%m-%d')
df['Order_Month'] = pd.to_datetime(df['Order_Date'].dt.strftime('%Y-%m'))
df.to_csv(r'C:\Users\Public\Downloads\SuperMart grorocery sales1.csv', index=False)
print(df.dtypes)
# Check for missing values
df.isnull().sum()
```

3. Exploratory Data Analysis (EDA):

```
# Statistical Analysis:
df.shape
df.describe()
```

```
# Visualization:

# Monthly Sales and Profit

df['Order_Date'] = pd.to_datetime(df['Order_Date'], format='%d-%m-%Y')

df['Order_Month'] = df['Order_Date'].dt.to_period('M').astype(str)

monthly_df = df.groupby('Order_Month').agg({'Sales': 'sum', 'Profit': 'sum'}).reset_index()

plt.figure(figsize=(8, 5))

plt.plot(monthly_df['Order_Month'], monthly_df['Sales'], linestyle='-', color='darkred', linewidth=2, label='Sales')

plt.plot(monthly_df['Order_Month'], monthly_df['Profit'], linestyle=':', color='seagreen', linewidth=2, label='Profit')

plt.title('Monthly Sales and Profit Analysis (2015-2018)', fontsize=14)

plt.xlabel('Month', fontsize=12)

plt.ylabel('Amount', fontsize=12)

plt.grid(which='major', linestyle='--', linewidth=0.7, alpha=0.7)

plt.xticks(monthly_df['Order_Month'][::6], rotation=45, fontsize=10)

plt.yticks(fontsize=10)

plt.legend(loc='upper left', fontsize=10)

plt.tight_layout()

plt.show()

#Top 10 Cities Category-Wise Sales

filtered_data = pd.read_csv(r"C:\Users\Public\Downloads\SuperMart grorocery sales1.csv")

top_cities = filtered_data.groupby('City')['Sales'].sum().nlargest(10).index

filtered_data = filtered_data[filtered_data['City'].isin(top_cities)]

city_order = sorted(filtered_data['City'].unique())

filtered_data['City'] = pd.Categorical(filtered_data['City'], categories=city_order, ordered=True)

custom_colors = {

'Bakery': '#2a9d8f',

'Beverages': '#bc6c25',

'Eggs, Meat & Fish': '#577590',

'Food Grains': '#c08497',

'Fruits & Veggies': '#7a9e9f',

'Oil & Masala': '#ddb967',
```

```
'Snacks': '#6A5ACD'
}
sns.set_style("whitegrid")
plt.figure(figsize=(8, 5))
sns.barplot(data=filtered_data, x='City', y='Sales', hue='Category', palette=custom_colors,ci=None)
plt.xlabel('City', fontsize=12)
plt.ylabel('Sales', fontsize=12)
plt.title('Top 10 Cities with Category-Wise Sales', fontsize='14')
plt.legend(title='Category', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
#Top 5 Customers by Frequency
customer_freq = df['Customer Name'].value_counts().reset_index()
customer_freq.columns = ['Customer Name', 'Count']
top_customers = customer_freq.head(5)
plt.figure(figsize=(8, 5))
colors = plt.cm.Paired(range(len(top_customers)))
wedges, texts, autotexts = plt.pie(
top_customers['Count'],
labels=top_customers['Customer Name'],
autopct='%1.1f%%',
startangle=90,
colors=colors,
wedgeprops={'linewidth': 1, 'edgecolor': 'white'},
pctdistance=0.85
)
centre_circle = plt.Circle((0, 0), 0.70, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title('Top 5 Customers by Frequency')
```

```
plt.legend(wedges, top_customers['Customer Name'], title="Customer Names", loc="center left", bbox_to_anchor=(1, 0, 0.5, 1))

plt.tight_layout()

plt.show()

#Top 10 Frequenty Sub_Category Combinations:

subcategories_per_order = df.groupby('Sales')['Sub_Category'].apply(list).reset_index()

combinations_list = []

for subcategories in subcategories_per_order['Sub_Category']:

if len(subcategories) >= 2:

combinations_list.extend(combinations(subcategories, 2))

if len(subcategories) >= 3:

combinations_list.extend(combinations(subcategories, 3))

combination_counts = Counter(combinations_list)

top_combinations = combination_counts.most_common(10)

if top_combinations:

top_combinations_df = pd.DataFrame(top_combinations, columns=['Combination', 'Frequency'])

top_combinations_df['Combination'] = top_combinations_df['Combination'].apply(lambda x: ' & '.join(x))

top_combinations_df.to_csv('top_combinations.csv', index=False)

print(top_combinations_df)

combos, counts = zip(*top_combinations)

combo_labels = [' & '.join(combo) for combo in combos]

sorted_combos = sorted(zip(combo_labels, counts), key=lambda x: x[1])

sorted_combo_labels, sorted_counts = zip(*sorted_combos)

plt.figure(figsize=(8, 5))

plt.barh(sorted_combo_labels, sorted_counts, color='Green')

plt.xlabel('Frequency')

plt.ylabel('Itemsets')

plt.title('Top 10 Most Frequent Sub_category Combinations')

plt.show()

else:

print("No valid combinations found. Please check the dataset or reduce the min support for combinations.")
```

4. Predictive Modeling:

```
#Monthly Sales Trend by Category:
df['Order_Month'] = pd.to_datetime(df['Order_Month'], errors='coerce')
df['Order_Month'] = df['Order_Month'].dt.to_period('M')
monthly_sales = df.groupby([df['Order_Month'], 'Category'])['Sales'].sum().unstack()
plt.figure(figsize=(8, 5))
monthly_sales.plot(kind='line', figsize=(12, 5))
plt.title("Monthly Sales Trend by Category")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend(title="Category")
plt.grid(True)
plt.show()
#Sales Forecasting for the Next 12 Months:
df['Order_Date'] = pd.to_datetime(df['Order_Date'], format='%d-%m-%Y')
df.set_index('Order_Date', inplace=True)
monthly_sales = df['Sales'].resample('ME').sum()
sarima_model = SARIMAX(monthly_sales, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
sarima_fit = sarima_model.fit()
print(sarima_fit.summary())
forecast_steps = 12
forecast_index = pd.date_range(monthly_sales.index[-1], periods=forecast_steps + 1, freq='ME')[1:]
forecast = sarima_fit.forecast(steps=forecast_steps)
plt.figure(figsize=(8, 5))
plt.plot(monthly_sales, label='Actual Sales', color='Green')
plt.plot(forecast_index, forecast, label='Forecast', color='Orange', linestyle='--')
plt.title('Sales Forecasting for the Next 12 Months')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend()
plt.show()
```

5. Model Development:

```
#Impact of Discount on Sales and Profit:
#Discount vs Sales:
features = df[['Discount', 'Sales']]
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(scaled_features)
cluster_order = df.groupby('Cluster')['Sales'].mean().sort_values().index
cluster_mapping = {old: new for new, old in enumerate(cluster_order)}
df['Cluster'] = df['Cluster'].map(cluster_mapping)
colors = {0: 'red', 1: 'blue', 2: 'green'}
plt.figure(figsize=(8, 5))
for c in df['Cluster'].unique():
 cluster_data = df[df['Cluster'] == c]
plt.scatter(cluster_data['Discount'], cluster_data['Sales'], color=colors[c], label=f'Cluster {c}')
plt.title('Discount vs Sales')
plt.xlabel('Discount')
plt.ylabel('Sales')
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()
#Discount vs Profit
profit_features = df[['Discount', 'Profit']]
scaler_profit = StandardScaler()
scaled_profit = scaler_profit.fit_transform(profit_features)
kmeans_profit = KMeans(n_clusters=3, random_state=42)
df['Profit_Cluster'] = kmeans_profit.fit_predict(scaled_profit)
profit_order = df.groupby('Profit_Cluster')['Profit'].mean().sort_values().index
profit_map = {old: new for new, old in enumerate(profit_order)}
df['Profit_Cluster'] = df['Profit_Cluster'].map(profit_map)
```

```
plt.figure(figsize=(8, 5))
for c in df['Profit_Cluster'].unique():
data = df[df['Profit_Cluster'] == c]
plt.scatter(data['Discount'], data['Profit'],label=f'Profit Cluster {c}', alpha=0.7)
plt.title("Discount vs Profit (Clusters)")
plt.xlabel("Discount")
plt.ylabel("Profit")
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()
# Factors Affecting Sales:
features = ['Discount', 'Profit', 'Category', 'Region']
target = 'Sales'
df_encoded = pd.get_dummies(df[features], drop_first=True)
X = df_encoded
y = df[target]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
coefficients = pd.Series(model.coef_, index=X.columns)
coefficients = coefficients.sort_values(key=abs, ascending=False)
print("\nTop Factors Influencing Sales:")
print(coefficients)
plt.figure(figsize=(8, 5))
sns.barplot(x=coefficients.values, y=coefficients.index, palette='viridis')
plt.title("Factors Affecting Sales")
plt.xlabel("Impact on Sales")
plt.ylabel("Features")
plt.grid(True)
plt.tight_layout()
plt.show()
```

6. Evaluation:

```
#Discount vs Sales
kmeans = KMeans(n_clusters=3)
kmeans.fit(df[['Discount', 'Sales']])
df['cluster'] = kmeans.labels_
score = silhouette_score(df[['Discount', 'Sales']], df['cluster'])
print(f"Silhouette Score for KMeans Clustering: {score:.2f}")
#Discount vs Profit
kmeans = KMeans(n_clusters=3)
kmeans.fit(df[['Discount', 'Profit']])
df['cluster'] = kmeans.labels_
score = silhouette_score(df[['Discount', 'Profit']], df['cluster'])
print(f"Silhouette Score for KMeans Clustering: {score:.2f}")
```